DocuProcess Cloud

Procesamiento asíncrono de recibos y facturas en AWS, desplegado desde un sandbox vacío con un solo comando.

Juan José De La Pava · Diego Polanco

Proyecto final, Infraestructura III
Universidad ICESI, 13 de noviembre de 2026

DOCUPROCESS

us-east-1 · AWS Academy sandbox

CloudFront + S3	portal
ALB	HTTP :80
EC2 Auto Scaling	2–4
RDS MySQL	Single-AZ
DynamoDB	2 tablas
SNS	2 tópicos
CloudWatch	7 alarmas

TOTAL **make up**

Terraform 12 módulos + CloudFormation
sin IAM nuevo · sin SSH · sin Lambda

Un usuario sube un recibo. Minutos después, los datos extraídos y un correo lo están esperando.

Portal web

HTML, CSS y JavaScript estáticos en S3, servidos por CloudFront. Registro, inicio de sesión, subida, historial y detalle.

API REST

FastAPI en EC2. Sesiones, roles, validación de la subida (PDF, PNG, JPEG hasta 5 MiB) y consulta de estado.

Worker

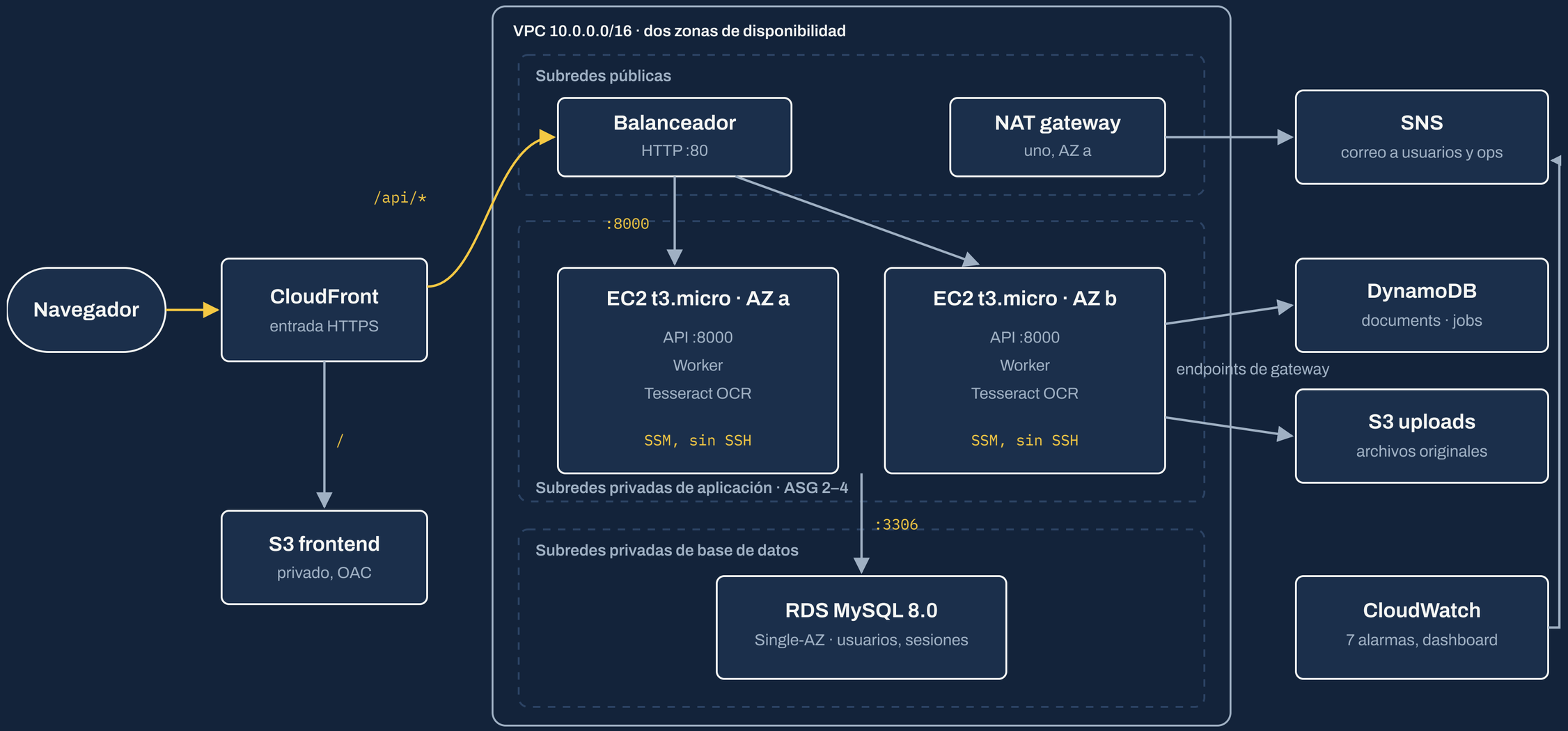
Corre junto a la API en cada instancia. Toma jobs, ejecuta el OCR con Tesseract y guarda el resultado en DynamoDB.

Notificaciones

Correo por SNS cuando un job termina o falla. Un segundo tópico lleva las alarmas de CloudWatch a los operadores.

Todos los documentos de prueba son sintéticos. El motor de extracción es una biblioteca Python pura, sin dependencia de AWS, así que se prueba sin la nube.

Arquitectura



La subida responde al instante. El trabajo ocurre después.

1. **Subida por CloudFront**
   La API verifica el tipo real del archivo y su tamaño, y responde 202 Accepted.
2. **Archivo a S3, job a DynamoDB**
   El job empieza en PENDING. La tabla jobs es la cola.
3. **Un worker lo toma**
   Una escritura condicional lo pasa a RUNNING con un lease de 300 segundos. Dos workers nunca toman el mismo job.
4. **OCR y extracción**
   Tesseract lee la página; el motor encuentra totales, fechas y moneda.
5. **Resultado guardado, correo enviado**
   El estado terminal queda en DynamoDB; SNS le envía al usuario el ID del documento y el estado.

ESTADOS DEL JOB

PENDING	en espera
RUNNING	lease 300 s
COMPLETED	todo extraído
PARTIAL	revisar
FAILED	código de error

recibo de prueba	receipt.pdf
TOTAL	**COP 119000.00**

Un lease vencido devuelve el job a otro worker.
El texto crudo del OCR nunca se guarda.

Todo es código. Un comando lo construye desde cero.

Una raíz de Terraform con doce módulos. Los cuatro buckets de S3 son un stack de CloudFormation que Terraform crea y destruye, porque el lab bloquea una lectura que necesita el recurso de S3 de Terraform.

network	security	alb
compute	database	dynamodb
buckets	bucket_policies	frontend
messaging	monitoring	audit

Un script de control rechaza el código si aparece cualquier recurso de IAM, función Lambda, regla de SSH o volumen gp3.

make up

unos 20 minutos

prepare-state	reconciliar
package	release .tar.gz
bootstrap-artifacts	buckets
publish	versión a S3
deploy	stack completo
deploy-frontend	portal + CDN
wait_healthy	2 targets sanos

SALIDA **https://…cloudfront.net**

Cada instancia se configura sola al arrancar.

El script de userData convierte una imagen limpia de Ubuntu 24.04 en una API y un worker. Nadie entra a configurarla, y una instancia reemplazada vuelve idéntica.

Log de arranque: `/var/log/docuprocess-bootstrap.log`

1. **Instalar paquetes**
   Python, Tesseract con datos de español e inglés, AWS CLI.
2. **Traer secretos y certificados**
   Contraseña de la base de datos desde SSM Parameter Store, CA de RDS.
3. **Descargar la versión**
   El archivo versionado desde el bucket de artefactos.
4. **Instalar la aplicación**
   Dependencias fijadas y tres paquetes en un entorno virtual.
5. **Migrar la base de datos**
   Por TLS verificado, con un lock de MySQL para que dos instancias nunca migren a la vez.
6. **Arrancar los servicios**
   systemd ejecuta la API y el worker; el health check del ALB se pone en verde.

Dos bases de datos, cada una con lo que mejor garantiza.

RDS MySQL: usuarios y sesiones

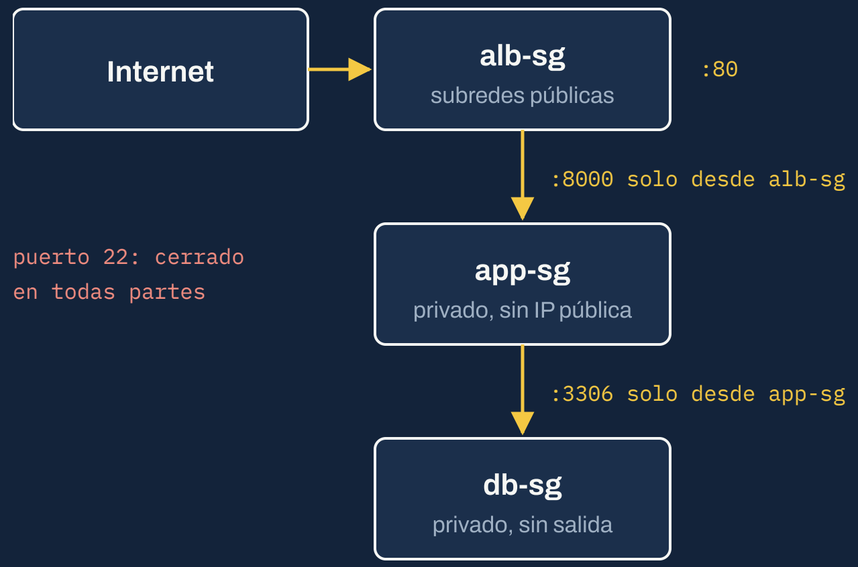
- Correo único y llave foránea de sesiones a usuarios, garantizados por el esquema.
- Tablas pequeñas y estables, consultas de administración ordenadas.
- Contraseñas como hashes Argon2id; tokens como hashes SHA-256.
- Costo aceptado: Single-AZ, la única opción en el sandbox.

DynamoDB: documentos, jobs, resultados

- Lecturas por llave: ID de documento o de job.
- Las escrituras condicionales hacen seguro tomar jobs desde varias instancias.
- Los resultados anidados cambian de forma sin migraciones.
- Capacidad bajo demanda para subidas en ráfagas.

Por qué no SQS: no está disponible en el lab, y con la tabla jobs como cola, el estado del job y la entrada de la cola son el mismo registro, así que nunca pueden contradecirse. Los leases, reintentos y esperas los implementamos nosotros.

Nada privado se puede alcanzar desde Internet.



Acceso de operadores

SSM Session Manager con el instance profile que ya existe en el lab. Sin llaves SSH ni bastión.

Permisos sin IAM nuevo

S3, DynamoDB y SNS le dan acceso al rol de la instancia con políticas de recurso, porque el lab prohíbe crear roles.

Datos en tránsito y en reposo

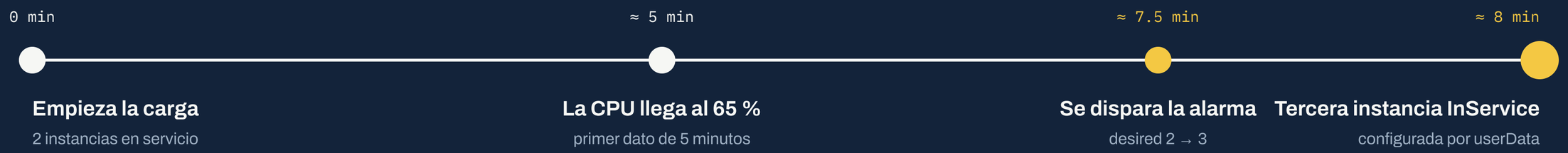
HTTPS en CloudFront, TLS hacia MySQL con el certificado verificado, SSE-S3 en todos los buckets.

Lo que los logs nunca contienen

Contenido de documentos, texto del OCR, correos ni montos. Solo IDs, códigos, conteos y tiempos.

Con carga, pasó de 2 a 3 instancias en ocho minutos.

Target tracking al 60 % de CPU promedio, entre 2 y 4 instancias. Observado el 2026-11-11 con `make load` y cuatro clientes iniciando y cerrando sesión.



Siete alarmas

CPU alta y pocas instancias en el ASG, hosts no sanos y 5xx en el ALB, CPU, almacenamiento libre y conexiones de RDS.

Llegan por correo

Cada alarma notifica al tópico SNS ops-alarms; un dashboard muestra las mismas métricas.

Auditoría

El lab niega crear trails, así que el historial de eventos de CloudTrail muestra cada llamada que hizo Terraform.

El sandbox dijo que no. Cada negativa se volvió una decisión.

Denegado o ausente	Qué se rompió	Qué hicimos
`iam:CreateRole`	No hay rol para las instancias	Usar el instance profile del lab; dar acceso con políticas de recurso
`s3:GetBucketObjectLock…`	Terraform no puede gestionar buckets de S3	Buckets en un stack de CloudFormation gestionado por Terraform
`volúmenes gp3`	RunInstances rechazado	Mapear el volumen raíz como gp2 en el launch template
`tesseract en AL2023`	Falló el arranque; ningún target sano, CloudFront 502	Cambiar a la imagen de Ubuntu 24.04 e instalar con apt
`StartInstanceRefresh`	Las versiones nuevas no se despliegan solas	Reemplazar instancias de a una, esperando a que estén sanas
`MonitorInstances`	CPU solo cada 5 minutos	Aceptar un escalado de 8 a 12 minutos; arrancar la carga temprano
`cloudtrail:CreateTrail`	No hay trail del proyecto	Usar el historial de eventos de CloudTrail; excepción documentada

Nuestra primera prueba de carga tumbó la API.

Lanzamos 24 clientes de inicio de sesión simultáneos. En lugar de escalar, el proceso de la API murió, el balanceador respondió 502 a casi todas las peticiones y el Auto Scaling Group reemplazó las instancias por no estar sanas.

La causa fue la memoria, no la CPU. Argon2id reserva 64 MiB en cada verificación de contraseña, por diseño. Una t3.micro tiene 1 GiB. Con cuatro clientes los inicios de sesión funcionan, la CPU llega a cerca del 89 % y el escalado se comporta como se esperaba.

CUENTA DE MEMORIA

t3.micro, una instancia

RAM	1024 MiB
Argon2id por login	64 MiB
24 clientes	502
4 clientes	CPU ≈ 89 %
POR DEFECTO	**4 clientes**

Limitar los intentos de login queda fuera de la v1: es un riesgo aceptado.

Un reset del lab borra casi todo. Casi.

Lo que encontramos

Tras un reset, el estado local seguía listando unos 64 recursos que ya no existían, y tres sobrevivientes bloqueaban un despliegue nuevo con errores “AlreadyExists”.

Los sobrevivientes

La contraseña de la base de datos en Parameter Store, un origin access control de CloudFront y una política de cabeceras de seguridad de CloudFront.

La solución

`make prepare-state` corre primero dentro de `make up`. Pone el instance profile nuevo, aparta el estado viejo e importa los sobrevivientes.

La VPC es el centinela: se crea primero y se destruye de última, así que si la VPC del estado ya no existe, todo el estado es de una sesión anterior. El archivo viejo se guarda como respaldo, nunca se borra.

Lecciones aprendidas

Probar la plataforma antes de diseñar sobre ella

Las verificaciones de solo lectura y dry-run encontraron la mayoría de las negativas antes de que rompieran un despliegue. Cada respuesta quedó escrita con su evidencia.

Que Terraform termine no significa que el sistema funcione

El primer despliegue aplicó sin errores y solo devolvía 502. Los health checks que prueban la base de datos son la señal real.

La seguridad tiene un costo de capacidad

Un hash de contraseñas fuerte hizo que el límite de una instancia pequeña fuera la memoria, no la CPU. Las pruebas de carga tienen que tenerlo en cuenta.

El estado es parte de la infraestructura

En un lab temporal, la desincronización es lo normal. Reconciliar el estado tiene que ser un paso del despliegue, no una reparación manual.

Las decisiones necesitan un registro

Treinta y ocho ADR permiten que cualquier integrante, o cualquier agente de IA, vea por qué se tomó una decisión en lugar de deshacerla.

Mantener el núcleo libre de la nube

El motor de extracción no importa nada de AWS, así que las 84 pruebas corren en segundos y sin credenciales.

Demostración en vivo

1. **Desplegar desde un sandbox limpio**
   `make up VERSION=1.0.0` y luego un segundo plan sin cambios.
2. **Mostrar la topología**
   Instancias privadas sin IP pública, dos targets sanos, una terminal por SSM.
3. **Procesar un documento**
   Subirlo en el navegador, seguir el job en DynamoDB, recibir el correo de SNS.
4. **Generar carga**
   `make load` y ver cómo sube la capacidad deseada.
5. **Mostrar alarmas y auditoría**
   El correo de la alarma de CPU, el dashboard, el historial de CloudTrail.

VERIFICACIÓN LOCAL

`make lint`	ok
`make typecheck`	ok
`make test`	84 ok
`make test-ocr`	PDF PNG JPEG
`make tf-validate`	válido

AL FINAL `make destroy`

Vacía todos los buckets y luego destruye el stack.

Preguntas

El repositorio contiene el código, las plantillas de Terraform y CloudFormation, los scripts, las especificaciones y 38 registros de decisiones. El plan y la guía de operación están en `entrega_proyecto_final/`.